

# CH4 - Clip 4.

## Container 실습



01

## 실습 소개

02

## 실습 환경 구축

# 1. 실습 소개

# Docker

## Docker란?

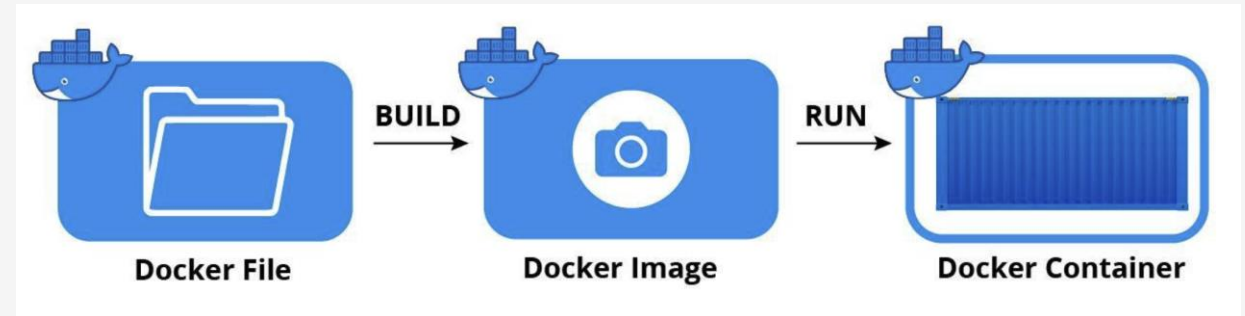
- **정의:** Docker는 컨테이너화 기술을 사용하여 애플리케이션을 패키징하고 배포하는 데 사용되는 오픈 소스 플랫폼. 이를 통해 애플리케이션을 쉽게 배포하고 실행할 수 있음
- **특징:** Docker는 경량화된 컨테이너를 제공하며, 이는 독립적이고, 이식성이 뛰어나며, 환경에 구애받지 않는 방식으로 애플리케이션을 실행



# Docker

## Docker 주요 구성 요소

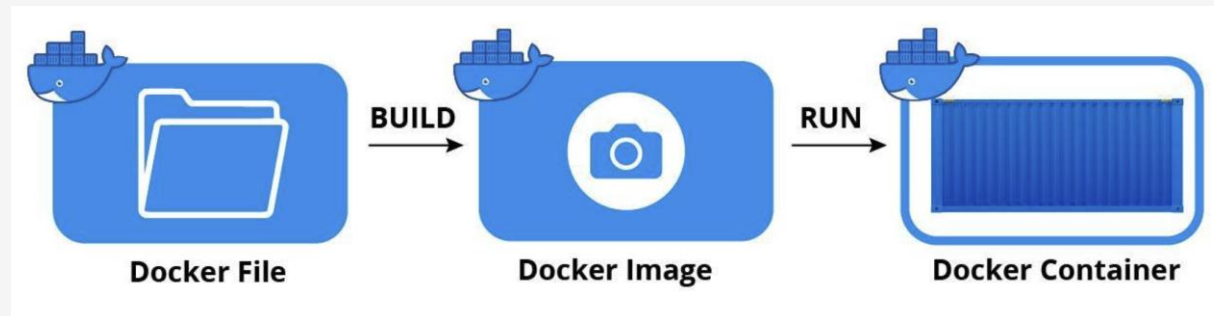
- **Docker Image**  
애플리케이션을 실행하는데 필요한 모든 파일과 설정을 포함하는 템플릿  
독립적으로 컨테이너가 실행됨
- **Docker Container**  
Image를 기반으로 실행되는 실행 Instance
- **Docker Daemon**  
Docker Image와 Container를 관리하는 background service
- **Docker Registry**  
외부 Image 저장소. 다른 사람들이 공유한 이미지를 사용할 수도 있고,  
private 하게도 저장 가능
- **Docker Client**  
사용자가 Docker Daemon과 상호작용하는 Interface



# Docker

## Docker 생성 단계

- 1단계 : Dockerfile 작성
- 2단계 : Dockerfile 내에 스크립트를 실행한다면, 스크립트를 작성
- 3단계 : Docker 이미지 빌드
- 4단계 : Docker 컨테이너 실행



# Docker

## Docker 생성 단계

- 1단계 : Dockerfile 작성
- 2단계 : Dockerfile 내에 스크립트를 실행한다면, 스크립트를 작성
- 3단계 : Docker 이미지 빌드
- 4단계 : Docker 컨테이너 실행

1단계 :

### Dockerfile

```
# 기본 이미지로 Python 3.8 사용
FROM python:3.8-slim
```

```
# 작업 디렉토리 설정
WORKDIR /app
```

```
# Python 스크립트 복사
COPY hello.py /app
```

2단계 :

```
# 스크립트 실행
CMD ["python", "./hello.py"]
```

3단계 : `docker build -t hello-world-python .`

4단계 : `docker run hello-world-python`

# Docker 실습

## Docker 생성 실습

- Nginx기반 서버 컨테이너 생성 및 실습
- Scikit-learn으로 모델 학습 컨테이너 생성 및 실습



## 2. 실습 환경 구축

## 실습 환경 구축

### 실습 환경 구축

- Docker Desktop 을 설치해서 Docker를 쉽게 관리할 수 있음

Window : <https://docs.docker.com/desktop/install/windows-install/>

Linux : <https://docs.docker.com/desktop/install/linux-install/>

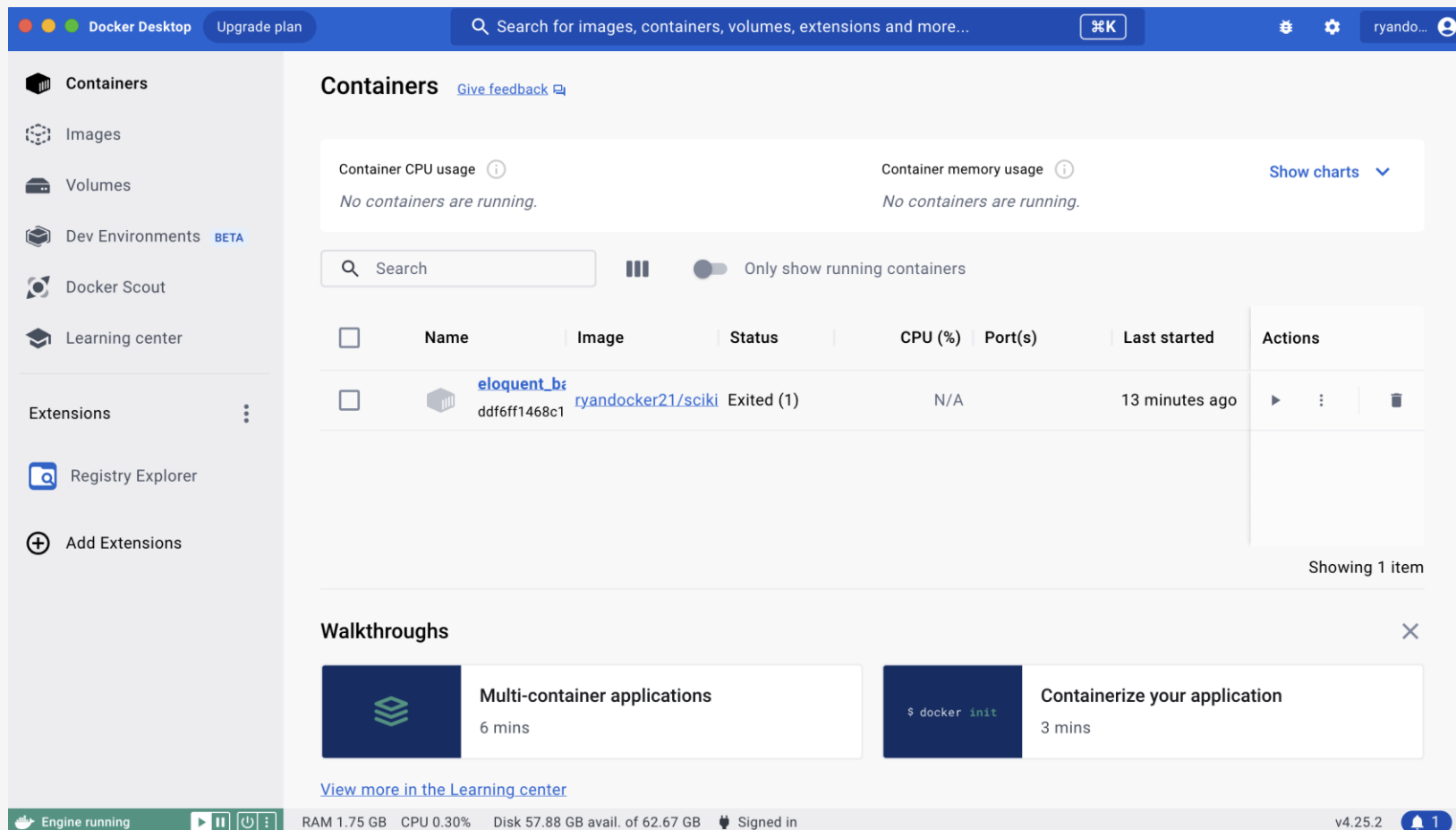
Mac : <https://docs.docker.com/desktop/install/mac-install/>

<https://labs.play-with-docker.com/>

# 실습 환경 구축

## 실습 환경 구축

- Docker Desktop 을 설치해서 Docker를 쉽게 관리할 수 있음



## 실습 환경 구축

### 실습 환경 구축

- 추후 실습을 위해 Docker Hub Account를 생성해주세요! (Free Account로 충분)

<https://hub.docker.com>

## 3. Hello Docker World!

# Hello Docker World

## 실습 진행

- 1단계 : Dockerfile 작성
- 2단계 : Dockerfile 내에 스크립트를 실행한다면, 스크립트를 작성
- 3단계 : Docker 이미지 빌드
- 4단계 : Docker 컨테이너 실행

1단계 :

### Dockerfile

```
# 기본 이미지로 Python 3.8 사용
FROM python:3.8-slim
```

```
# 작업 디렉토리 설정
WORKDIR /app
```

```
# Python 스크립트 복사
COPY hello.py /app
```

2단계 :

```
# 스크립트 실행
CMD ["python", "./hello.py"]
```

3단계 : `docker build -t hello-world-python .`

4단계 : `docker run hello-world-python`

# Hello Docker World

## 실습 진행

- Python version 추가 확인

1단계 :

```
Dockerfile

Import sys

...

print(sys.version_info)

...
```

2단계 : `docker build -t hello-world-python .`

3단계 : `docker run hello-world-python`

# Hello Docker World

## 실습 진행

- `docker ps`
- `docker rmi [docker id] -f`



## 4. 서버 Docker Container

# 서버 Docker Container

## 실습 진행

- 1단계 : Docker Registry 확인 및 검색
- 2단계 : docker login
- 3단계 : docker search nginx
- 4단계 : docker pull nginx
- 5단계 : docker images ls
- 6단계 : docker run -it -d -p 8001:80 -name nginxserver -p 8001:80 nginx:latest &

# Docker Registry Search

## 실습 해보기

- Docker exec로 해당 container bash 접속해서, index.html 파일 변경하기

첫번째 방법

- docker ps
- docker exec -it [docker id] bash
- cd /usr/share/nginx/html
- echo fastcampus' nginx server > index.html

두번째 방법

- docker cp index.html webserver:/usr/share/nginx/html/index.html

## 5. Scikit-learn docker container

# Docker Registry

## 주요 기능

- 이미지 저장 및 관리 : Docker 이미지를 업로드하고, 버전 관리하며 다운로드 할 수 있음
- 접근 제어 : 공개 또는 사설 저장소를 설정하여 이미지에 대한 접근을 제어할 수 있음
- 이미지 공유 : 팀원들과 이미지 공유하거나, 전 세계와 공개적으로 이미지를 공유할 수 있음
- 자동화된 빌드 : GitHub와 같은 소스 코드 저장소와 통합하여 이미지 빌드를 자동화 할 수 있음

## 주요 CLI

- 이미지 검색 : `docker search [TERM]`
- 이미지 태그 지정 : `docker tag [SOURCE_IMAGE] [REGISTRY_HOST]/[REPOSITORY]/[IMAGE]:[TAG]`
- 이미지 푸시 : `docker push [REGISTRY_HOST]/[REPOSITORY]/[IMAGE]:[TAG]`
- 이미지 풀 : `docker pull [REGISTRY_HOST]/[REPOSITORY]/[IMAGE]:[TAG]`
- 로그인 : `docker login [REGISTRY_HOST]`
- 로그아웃 : `docker logout [REGISTRY_HOST]`

# Scikit-learn Docker Container

## 실습 진행

- 1단계 : Dockerfile 작성
- 2단계 : python script 작성
- 3단계 : Docker 이미지 빌드 -> `docker build -t [image name]`
- 4단계 : Docker 컨테이너 실행 -> `docker run --name [your docker name] [your docker image:tag ]`
- 5단계 : docker image tag 변경 -> `docker tag [your docker image:tag on local] [your_hub_id/docker image:tag]`
- 6단계 : docker image를 registry에 저장 -> `docker push [your_hub_id/docker image:tag]`
- 7단계 : 로컬에 있는 docker image 삭제 -> `docker rmi -f [your docker image]`
- 8단계 : registry에 저장된 image를 로컬로 다운로드 받아서 실행 -> `docker pull [your_hub_id/docker image:tag]`